Computational Economics

Lecture: Q Functions, Optimality, and Value Function Iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **Action Values: The Q Function**
  - Fixing the first action
  - How Q^σ and V^σ are related
- **The Principle of Optimality**
  - Optimal continuation from the next state
  - Bellman optimality equations and branching diagrams
- **Value Function Iteration**
  - The Bellman update, convergence, and a stopping rule
  - Recovering a policy; job search and continuous saving

Recap: Returns and State Values

From state $s_t = s$, follow a stationary policy σ:

$$G_t = \sum_{\tau=1}^{\infty} \beta^{\tau-1} r_{t+\tau}, \qquad V^{\sigma}(s) = \mathbb{E}_{\sigma}[G_t \mid s_t = s].$$

- $V^{\sigma}(s)$ evaluates a **state** when every action follows σ.
- Realized actions, rewards, and next states come from σ, R, and P. The dynamics are time-homogeneous.
- Assume $0 < \beta < 1$ and bounded absolute rewards. The return and value are then finite.
- We first use finite states and actions. The saving example later has a continuous state and action space.

Example: Which First Move?

At S, compare **up** with **down**, then follow the same continuation policy σ.

- Both moves give reward -1.
- Their next states differ, so their future returns can differ.
- $V^{\sigma}(\mathrm{S})$ gives one number for S. We need a value for **each first action**.

G is absorbing; rewards are zero there. Every move before reaching G costs one, including entry into G.

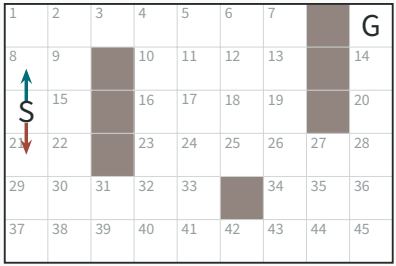


Figure 1: Two choices at S.

The Action-Value Function

The **action-value function** $Q^\sigma(s, a)$ is the expected return when we start at s, take action a, and follow σ thereafter:

$$Q^\sigma(s, a) = \mathbb{E}_\sigma \left[\sum_{\tau=1}^{\infty} \beta^{\tau-1} r_{t+\tau} \,\middle|\, s_t = s,\ a_t = a \right].$$

- The first action a is **fixed**; actions from date $t+1$ onward follow σ.
- a may differ from the action that σ would choose at s. Interpret the conditioning as deliberately taking a, even if σ never selects it.
- Rewards and subsequent states can still be random.

Linking Q and V

Fixing the first action gives

$$Q^{\sigma}(s,a) = \mathbb{E}[r_{t+1} + \beta V^{\sigma}(s_{t+1}) \mid s_t = s, a_t = a].$$

For discrete rewards and next states, expand this expectation:

$$Q^{\sigma}(s,a) = \sum_{r} \Pr\{R(s,a) = r\} \sum_{s'} P(s' \mid s,a,r)\big[r + \beta V^{\sigma}(s')\big].$$

Averaging the first action according to the policy recovers the state value:

$$V^{\sigma}(s) = \sum_{a \in \mathcal{A}(s)} \Pr\{\sigma(s) = a\}\, Q^{\sigma}(s,a).$$

For a deterministic policy, $V^{\sigma}(s) = Q^{\sigma}(s, \sigma(s))$.

The conditional kernel allows reward–next-state dependence. Replace sums by probability integrals for continuous outcomes.

Why Use Q?

- **Compare actions at the same state.** $Q^\sigma(s, a)$ includes the immediate reward and the consequences of following σ afterward.
- **Choose an action directly.** Given action values, a **greedy action** is

$$a_g(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^\sigma(s, a).$$

- **Learn from experience.** Returns from rollouts starting with (s, a) can estimate $Q^\sigma(s, a)$; an explicit transition model is not required for this estimation.

A greedy choice from Q^σ uses σ's continuation values. It need not already be optimal. Storing action values also takes more space than storing one value per state.

Example: An Optimal Continuation

Keep the route from S to cell 23 fixed. At cell 23, compare two continuations to G.

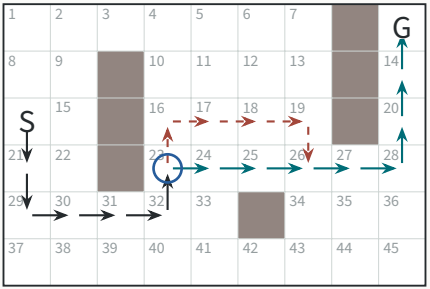


Figure 2: A common prefix and two continuations.

- Solid continuation: 8 moves.
- Dashed detour: 10 moves, including the shared final segment.

At cell 23, a continuation of L moves gives

$$-\sum_{j=0}^{L-1} \beta^j.$$

Replacing the detour raises return from S by $\beta^6(\beta^8 + \beta^9) > 0$.

The Principle of Optimality

For an optimal policy, the continuation must be optimal from each state reached with positive probability.

- Suppose a better continuation is available at a reached state.
- Keep all earlier choices unchanged and switch to that better continuation.
- The original plan's expected return then increases, contradicting its optimality.

The **Markov state** contains the information needed for future decisions. Earlier rewards are already realized; they do not change which continuation is best.

With uncertainty, the continuation is a policy for every possible next state. Optimality concerns **expected return**, not whether each realized path is the shortest or luckiest.

Optimal State and Action Values

For a finite discounted MDP, an optimal stationary deterministic policy exists. Define

$$V^*(s) = \max_\sigma V^\sigma(s), \qquad Q^*(s,a) = \max_\sigma Q^\sigma(s,a).$$

The principle of optimality gives

$$\begin{aligned} Q^*(s,a) &= \mathbb{E}[r_{t+1} + \beta V^*(s_{t+1}) \mid s_t = s, a_t = a], \\ V^*(s) &= \max_{a \in \mathcal{A}(s)} Q^*(s,a). \end{aligned}$$

- Q^*: fix the current action, then choose optimally afterward.
- V^*: also choose the current action optimally.
- The expectation averages the outcomes of the chosen action; it does not let us choose the random outcome.

Optimality as a One-Step Look Ahead

$$V^*(s) = \max_a Q^*(s, a)$$

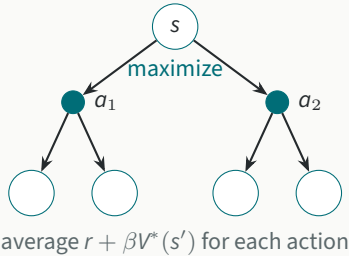


Figure 3: Choose now, average outcomes.

$$Q^*(s, a) = \mathbb{E}[r + \beta \max_{a'} Q^*(s', a') \mid s, a]$$

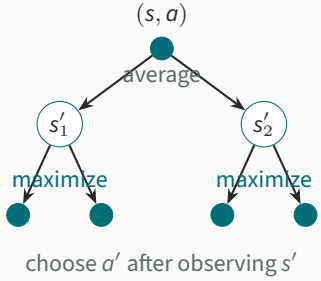


Figure 4: Fix now, choose at the next state.

Open circles: states. Filled circles: state–action pairs. Adapted from Sutton and Barto (2018), Figure 3.4; branches group reward and next-state outcomes.

The Bellman Optimality Operator

An **operator** maps a candidate value function v to a new function:

$$(Tv)(s) = \max_{a \in \mathcal{A}(s)} \sum_{r} \Pr\{R(s,a) = r\} \sum_{s'} P(s' \mid s, a, r)\big[r + \beta v(s')\big].$$

- $v(s')$ is our current estimate of the continuation value.
- For each action, average current reward plus discounted continuation value; retain the largest result.
- The optimal value is a **fixed point**: $TV^* = V^*$.
- Policy evaluation keeps a given policy fixed. This operator maximizes over actions at every update.

The sums describe discrete outcomes; probability integrals give the continuous version.

Value Function Iteration

For a finite MDP with known reward and transition distributions:

1. Initialize $v_0(s) = 0$ for all states; keep absorbing zero-reward terminal states at zero.
2. For every state s, score each feasible action using the **old** function:

$$q_k(s,a) = \sum_r \Pr\{R(s,a) = r\} \sum_{s'} P(s' \mid s,a,r)\left[r + \beta v_k(s')\right].$$

3. Store the best score in a new function: $v_{k+1}(s) = \max_a q_k(s,a)$.
4. Check the stopping rule; otherwise replace v_k by v_{k+1} and repeat.

Here q_k is an action score based on the current approximation; it need not equal Q^σ for a particular policy.

One **sweep** updates every state once. Using a separate new array makes these updates synchronous.

Example: Values Move Back Through the Maze

Consider cells 28, 20, and 14 near G in the same maze. Set $\beta = 0.9$, $v_0 = 0$; each move before G costs one.

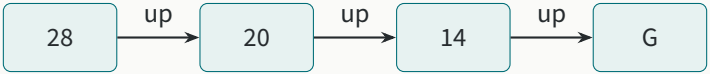


Figure 5: The final corridor to G.

	Cell 28	Cell 20	Cell 14	G
v_0	0	0	0	0
v_1	-1	-1	-1	0
v_2	-1.9	-1.9	-1	0
v_3	-2.71	-1.9	-1	0

For example, $v_3(28) = -1 + 0.9 v_2(20) = -2.71$. With zero initialization, v_k is the optimal k-period return with zero value afterward.

Why VFI Converges

For finite states, use $\|v - w\|_\infty = \max_s |v(s) - w(s)|$. The difference between two scores for the **same action** satisfies

$$|\beta \mathbb{E}[v(s') - w(s') \mid s, a]| \leq \beta \|v - w\|_\infty.$$

Since $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$,

$$\|Tv - Tw\|_\infty \leq \beta \|v - w\|_\infty.$$

Thus T is a **contraction**: it reduces the maximum distance by at least the factor $\beta < 1$.

- There is a unique fixed point V^*; every initial finite vector converges to it.
- After k sweeps,
$$\|v_k - V^*\|_\infty \leq \beta^k \|v_0 - V^*\|_\infty.$$
- A discount factor closer to one gives a slower worst-case rate.

A Stopping Rule With an Error Bound

After computing $v_{k+1} = Tv_k$, measure the **Bellman residual** of v_k:

$$d_k = \|Tv_k - v_k\|_\infty = \|v_{k+1} - v_k\|_\infty.$$

The contraction inequality gives

$$\begin{aligned}\|v_k - V^*\|_\infty &\le \|v_k - Tv_k\|_\infty + \|Tv_k - TV^*\|_\infty \\ &\le d_k + \beta\|v_k - V^*\|_\infty,\end{aligned}$$

so $\|v_k - V^*\|_\infty \le d_k/(1-\beta)$.

- To guarantee value error at most ε, stop when $d_k \le (1-\beta)\varepsilon$ and return v_k.
- At $\beta = 0.9$ and $\varepsilon = 0.01$, require $d_k \le 0.001$.

This guarantee uses exact expectations and maximization. Numerical approximation introduces additional errors.

Recovering a Policy

For the returned value estimate v, choose

$$\widehat{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} \sum_{r} \Pr\{R(s,a) = r\} \sum_{s'} P(s' \mid s, a, r) \left[r + \beta v(s') \right].$$

- With $v = V^*$, any maximizing action is optimal; ties are allowed.
- With an approximate v, this is a candidate policy. Evaluate it to check the return it actually achieves.
- Greedy means maximizing **current reward plus continuation value**, not just immediate reward.
- Recompute action scores using the returned v, and break ties consistently.

Example: VFI for McCall Job Search

The state is (U, w) or (E, w). Rejecting pays b and draws a new offer w' with probability $q(w')$; accepting pays w forever.

$$v_k((\mathrm{E}, w)) = \frac{w}{1-\beta} \quad \text{for every } k.$$

Use this known employed value and update unemployed states:

$$\begin{aligned} q_k((\mathrm{U}, w), \text{accept}) &= \frac{w}{1-\beta}, \\ q_k((\mathrm{U}, w), \text{reject}) &= b + \beta \sum_{w'} q(w') v_k((\mathrm{U}, w')), \\ v_{k+1}((\mathrm{U}, w)) &= \max\{q_k((\mathrm{U}, w), \text{accept}),\ q_k((\mathrm{U}, w), \text{reject})\}. \end{aligned}$$

- Start unemployed-state values at zero; keep employed-state values fixed at their exact values.
- At convergence, accept when the acceptance score is at least the rejection score.

Lowercase $q(w')$ is the offer probability; $q_k(s, a)$ is the iteration's action score.

Example: McCall Iterations

Use $\beta = 0.9$, $b = 1$, offers $(1, 2, 3, 4, 5)$, and probabilities $(0.1, 0.2, 0.4, 0.2, 0.1)$.

	$w = 1$	$w = 2$	$w = 3$	$w = 4$	$w = 5$
v_0	0.00	0.00	0.00	0.00	0.00
v_1	10.00	20.00	30.00	40.00	50.00
v_2	28.00	28.00	30.00	40.00	50.00
v_3	31.06	31.06	31.06	40.00	50.00
v_10	34.20	34.20	34.20	40.00	50.00
V^*	34.32	34.32	34.32	40.00	50.00

Columns give unemployed-state values; the last row is V^*. Acceptance values are $(10, 20, 30, 40, 50)$.

- The first update values waiting at only 1; later updates incorporate the value of future offers.
- In the limit, reject offers 1–3 and accept 4–5. For rejected offers, the common value solves

$$x = 1 + 0.9\big(0.7x + 0.2 \times 40 + 0.1 \times 50\big), \qquad x = \frac{12.7}{0.37}.$$

Example: VFI for Continuous Saving

Stock $s \in [0, M+B]$; save $a \in [0, \min(s, M)]$, consume $s-a$, and draw $Y' \sim \text{Uniform}[0, B]$ independently. Next stock is $s' = a + Y'$. Utility is c^α, with $0 < \alpha < 1$, $M \geq 0$, and $B > 0$.

$$
\begin{aligned}
q_k(s,a) &= (s-a)^\alpha + \frac{\beta}{B}\int_0^B v_k(a+y)\,dy, \\
v_{k+1}(s) &= \max_{a \in [0, \min(s,M)]} q_k(s,a), \\
\widehat{\sigma}(s) &\in \operatorname*{arg\,max}_{a \in [0, \min(s,M)]} \left\{ (s-a)^\alpha + \frac{\beta}{B}\int_0^B v(a+y)\,dy \right\}.
\end{aligned}
$$

- Every saving choice trades current consumption against future resources.
- The exact operator is a contraction on bounded continuous functions under the supremum norm. Rewards are bounded and feasible intervals are compact.

Computing the Continuous Update

The saving model has infinitely many states and actions. A computer needs a finite representation:

1. Store values at selected stock points and **interpolate** between them.
2. Approximate the income integral by a weighted average, using **numerical quadrature** or sampled incomes.
3. Maximize over the saving interval using numerical optimization or a feasible action grid.

The model remains continuous. A small iteration residual measures convergence of the numerical update; it does not by itself establish accuracy for the original model.

Refine the state representation, integration, and maximization to check that values and choices are stable.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Section 3.5: Policies and Value Functions
   - Section 3.6: Optimal Policies and Optimal Value Functions
   - Section 4.4: Value Iteration

2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — The Bellman Equation and the Principle of Optimality
   - Discrete State Dynamic Programming — Value Function Iteration
   - Job Search I: The McCall Search Model — The McCall Model
   - Discrete State Dynamic Programming — Example: A Growth Model
   - The Income Fluctuation Problem I: Basic Model — The Optimal Savings Problem